

Smartphone GNSS Spoofing Effects on Positioning and Receiver States

Filippo Ferrari, Sibilla Merlo, Flavio Carpinteri, Francesco Pingaro

Politecnico di Torino

ABSTRACT

This report analyzes the laboratory material on Android GNSS measurements with focus on three required tasks: nominal data inspection, spoofed-position emulation, and spoofing delay injection.

1 METHODS

The analysis follows the laboratory workflow described in the course handout: GNSS raw measurements are acquired with a commercial Android device (Samsung Galaxy S9) through GNSS Logger [2] and post-processed with the `gps-measurement-tools` MATLAB scripts [1]. Background on Android raw GNSS measurements and their interpretation follows the reference material by van Diggelen and collaborators [4] and the material provided during the Wireless Security Laboratory lecture[3]. During the spoofing experiments, an additional robustness fix was required in `emulateSpoofing.m`; otherwise the script crashed on late epochs when the spoof-difference vector was shorter than the set of tracked satellites (see Appendix B).

The available material contains four relevant scenarios built from the same experimental campaign:

- **Home**: post-processing of a static, indoor, GNSS Logger acquisition;
- **Home Battery Save**: static acquisition with Android power-saving enabled, to assess the impact of receiver/clock management on tracking continuity;
- **Home Spoofed**: software spoofing enabled starting at `spoof.t_start=15 s`, with no added spoofing delay and target counterfeit position `spoof.position=[45.318150, 8.414489, 213.22]`. This target was chosen to be close to the nominal solution (near the median of the static **Home** run);
- **Home Spoofed + Delay**: same counterfeit position, plus `spoof.delay=40.212 ms`.

The discussion uses only figures that directly support the required conclusions. In particular, we considered the WLS position plots, geoplots, hardware-clock indicators, common-bias plots, and pseudorange derivative plots.

2 RESULTS AND DISCUSSION

2.1 Baseline collection and battery-saving effects

The nominal home dataset remains geographically coherent. Its WLS median is reported at $[45.315976^\circ, 8.416836^\circ]$ and the radius containing the best 50% of estimates is 22.6 m (see Figure 1). In the same run, $\text{diff}(\text{raw pr})/\text{diff}(\text{time})$ remains close to

the reported pseudorange-rate trends for the tracked satellites (2, 17, 19, 22, 32), with small mean mismatches ($[-0.04, -0.02, -0.46, 0.19, -0.08]$ m/s from the plot annotation) (see Figure 2). This supports that the baseline log is internally consistent for discussing spoofing-induced changes.

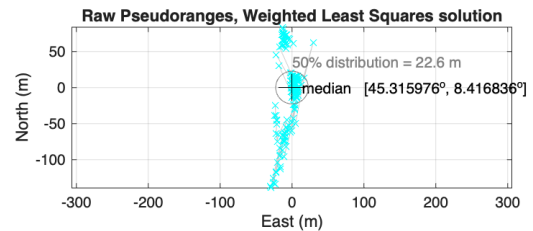


Figure 1: Nominal home WLS solution. The estimate cloud remains compact around the reported median.

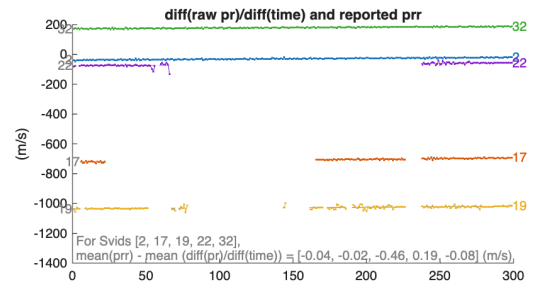


Figure 2: Nominal home pseudorange-rate consistency check ($\text{diff}(\text{raw pr})/\text{diff}(\text{time})$ vs. reported PRR). Small offsets indicate internally consistent measurements in the baseline log.

In the **battery-saving run**, the behavior changes mainly in continuity and visibility rather than in one single position metric. The HW clock discontinuity timeline (Figure 3) shows fragmented availability windows, and only subsets of satellites are visible in each interval. The C/N_0 plot (Figure 4) confirms this: one satellite (SV19) decays from mid-20 dB Hz values down to about 14 dB Hz, while later windows are dominated by SV2/SV17 around 24 dB Hz to 28 dB Hz. Therefore, in this updated dataset the key battery-saving signature is reduced tracking continuity and changing satellite subsets, not a single catastrophic position blow-up.



Figure 3: Battery-save HW clock discontinuity. The receiver clock state is not continuously available, leading to fragmented epochs and reduced tracking continuity.

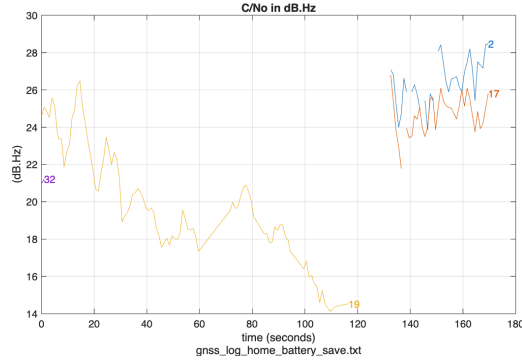


Figure 4: Battery-save C/N_0 evolution. Tracked satellites change over time and SV19 shows a progressive reduction in signal strength, consistent with unstable tracking conditions.

2.2 Spoofed position without delay

The spoofed-position experiment clearly demonstrates a successful takeover of the receiver's estimated navigation solution. Immediately after the attack initiates at the $t = 15$ s mark, the WLS median abruptly shifts from the authentic home baseline to the new coordinates $[45.318138^\circ, 8.414505^\circ]$. This represents a significant 301.7 m displacement, landing just 1.8 m from the attacker's configured target of $[45.318150^\circ, 8.414489^\circ]$.

While the spoofed estimate cloud remains localized, it exhibits a broader dispersion than the nominal case, with the 50% distribution radius widening from 22.6 m to 33.6 m (see Figure 6). The geo-plot visually confirms this with a stark split between the authentic pre-spoof points and the counterfeit post-spoof cluster (Figure 5). Crucially, this coherent displacement of hundreds of meters occurs without a collapse in measurement quality indicators, such as the C/N_0 levels. The system tracks the counterfeit signals as if they were genuine. This highlights a practical detection strategy: a robust monitor cannot rely on C/N_0 alone; instead, it must correlate three symptoms: **a sudden position shift, synchronized steps in multiple pseudoranges around the attack start time, and a mismatch between large navigation changes and stable signal-strength statistics**. Ultimately, a *massive position jump lacking corresponding RF-quality degradation* is the clearest signature of this attack.

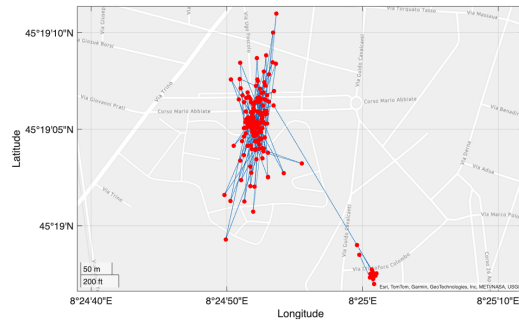


Figure 5: Geoplot for spoofed-position case with no added delay. Baseline and spoofed clusters are clearly separated in map space.

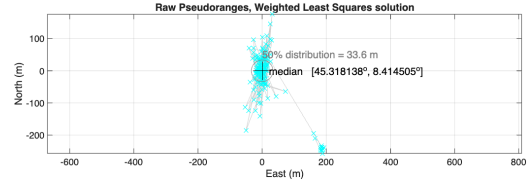


Figure 6: WLS scatter for spoofed-position case with no added delay. The solution remains concentrated around the counterfeit median.

2.3 Added spoofing delay

The next step of the experiment was to introduce a spoofing delay and evaluate whether it affects the estimated position and which additional artifacts it produces. In the updated plots, the spoofed-plus-delay solution has a median of $[45.318109, 8.414492]$, about 299.7 m from the baseline and about 3.4 m from the no-delay spoofed median. Hence, the estimate remains in the same counterfeit area, and the dispersion is comparable: the 50% radius is 31.4 m versus 33.6 m without delay (see Figure 7).

The strongest change is not horizontal position, but timing-related states: the derivative plot (Figure 8) exhibits a dominant spike ($\sim 1.2 \times 10^7$ in plot scale), and the common clock bias jumps to about 1.2×10^7 m (see Figure 9). This matches the expected effect of adding 40.212 ms delay, equivalent to roughly 12 055 km of apparent propagation range (see Appendix A). The receiver absorbs most of this perturbation into clock-like states, while the counterfeit position remains geographically similar.

This behavior is coherent with GNSS observation models: a *common* delay term is largely indistinguishable from receiver clock bias, because it shifts all pseudoranges in the same direction. For this reason, delay can be used as a component of a more subtle attack strategy. In principle, an attacker may try to inject a gradual common delay to mimic an abnormal clock drift and avoid an immediate position jump, then combine it with satellite-dependent terms to guide position progressively. **In our dataset, the applied delay is large enough to create a clear transient, so it is not stealthy.**

The magnitude of the delay directly impacts the *detectability*. If the delay is smaller, the induced bias may stay closer to natural clock-noise levels and become harder to separate from normal receiver behavior (especially in noisy or mobile conditions). If the delay is larger, common-bias and pseudorange-derivative anomalies become much more visible, potentially triggering consistency checks even when map-level position does not move much. Therefore, from a defensive perspective, monitoring the temporal profile of common bias (not just final position) is essential to capture both abrupt and progressive delay-based spoofing attempts.

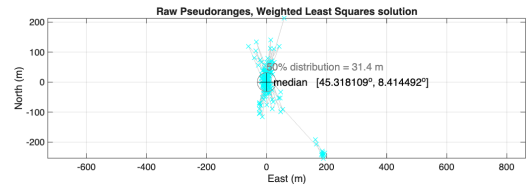


Figure 7: Delay case: WLS. Solution remains near the same counterfeit area.

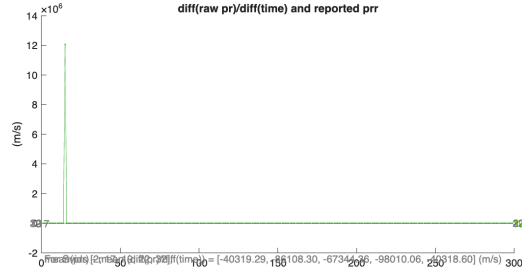


Figure 8: Delay case: derivative impulse. The added delay produces a dominant transient in pseudorange-derivative behavior.

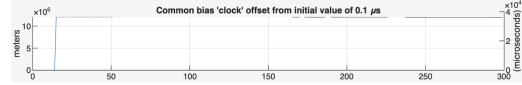


Figure 9: Delay case: clock-bias jump. The common-bias state rises to a much larger level than in no-delay spoofing.

2.4 Optional peculiar condition: outside moving dataset and hardware anomalies

To evaluate natural signal degradation versus intentional spoofing, an optional moving outdoor log was analyzed. Although not subjected to spoofing, this dataset exhibits an extremely broad spatial dispersion. As illustrated in the Geoplot (Figure 10), the WLS median position is $[45.315248^\circ, 8.374133^\circ]$, located approximately 3.3 km from the static home baseline, with a massive 50% error distribution radius of 3337.4 m. This vast spread demonstrates that user motion, dynamic satellite visibility, and environmental multi-path can naturally generate severe position scattering.

The corresponding panel C / N_0 (Figure 11) confirms a richer but highly unstable tracking context: signal strengths fluctuate widely from high teens to mid-40 dB-Hz. Under these kinematic conditions, a large geometric spread is expected and must not be misclassified as spoofing. The defining contrast is that natural kinematic spread is accompanied by continuous, random multi-satellite visibility dynamics, whereas intentional spoofing exhibits an abrupt, coordinated displacement after a specific transition time.

Furthermore, the enormous magnitude of the spatial outliers in this specific log necessitates a practical hypothesis regarding the test device’s hardware age. We postulate that the anomalous spread was drastically amplified by sudden battery voltage drops during the acquisition. Because the Weighted Least Squares (WLS) algorithm must resolve four unknowns (3D position and receiver clock bias), any corruption in the time-stamping of a satellite signal due to power instability is absorbed directly into the equations. This justifies the massive, non-stationary drift observed in the *Common bias 'clock' offset* (Figure 12).

Interestingly, the *Hardware Clock Discontinuity* indicator remains on “continuous”, indicating that the Android OS never officially triggered a duty-cycling reset. Therefore, while battery degradation cannot be identified as the root cause by the OS flags alone, the mathematical response of the WLS algorithm to the corrupted pseudoranges makes it a highly plausible contributing factor to the extreme PVT anomalies.

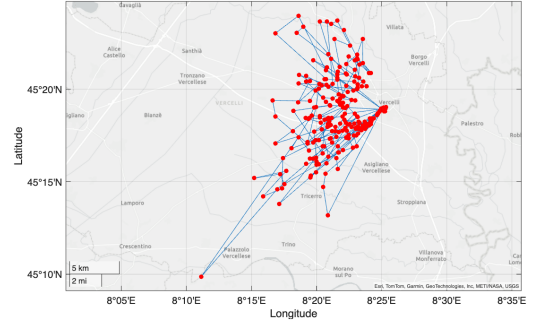


Figure 10: Outside-moving geoplex. Regional-scale spread is visible during motion.

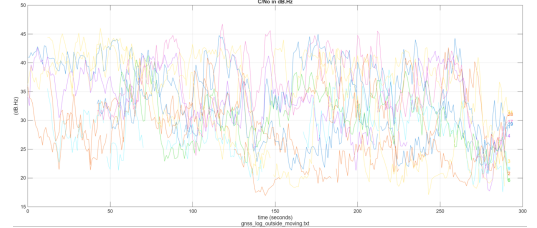


Figure 11: C/ N_0 fluctuations highlighting a rich but unstable tracking context.

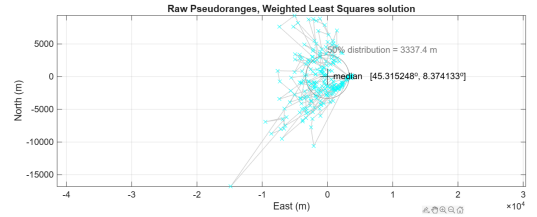


Figure 12: Common clock bias offset showing massive non-stationary drift.

3 CONCLUSION

This report evaluated a commercial Android GNSS receiver under nominal, power-constrained, and spoofed conditions. We demonstrated that while battery-saving modes disrupt tracking continuity via hardware duty-cycling, they do not cause catastrophic positioning failure. In contrast, intentional zero-delay spoofing induces an abrupt spatial translation (≈ 300 m) without degrading the C/N_0 signal strength.

Crucially, injecting a temporal delay (40 ms) does not alter the counterfeit spatial footprint but is absorbed by the WLS algorithm as a massive receiver clock bias anomaly ($\approx 1.2 \times 10^7$ m), generating severe spikes in the pseudorange rates. Furthermore, while natural kinematic degradation (e.g., the outside moving dataset) can produce broad spatial dispersion similar to an attack, it is uniquely characterized by continuous multi-satellite tracking instability and erratic C/N_0 levels, unlike the coordinated, abrupt transitions of intentional spoofing.

Future Work: Future experiments should evaluate dual-frequency (L1/L5) smartphones to assess whether multi-band tracking inherently improves positioning resilience and spoofing detection compared to traditional single-frequency measurements.

REFERENCES

- [1] Google. 2026. `gps-measurement-tools`. <https://github.com/google/gps-measurement-tools>. (2026). GitHub repository, accessed 2026-03-27.
- [2] Mohammed Khider. 2017. GnssLogger v2.0.0.1 Release. <https://github.com/google/gps-measurement-tools/releases/tag/2.0.0.1>. (2017). Release page.
- [3] Andrea Nardin. 2026. Wireless Security Lab: Spoofing Android GNSS Measurements. (2026). Course laboratory slides.
- [4] Frank van Diggelen and collaborators. 2026. Using GNSS Raw Measurements on Android Devices. https://www.gsc-europa.eu/sites/default/files/sites/all/files/gnss_raw_measurement_web_0.pdf. (2026). GSA/Google white paper.

A DELAY-TO-RANGE EQUIVALENCE

GNSS pseudorange is fundamentally a time-of-flight measurement. A delay Δt applied to the received signal is equivalent to an additional apparent propagation distance $\Delta \rho$ given by:

$$\Delta \rho = c \Delta t$$

where $c \approx 299\,792\,458$ m/s is the speed of light.

For the spoofing delay used in the experiment, $\Delta t = 40.212$ ms = 40.212×10^{-3} s, thus:

$$\Delta \rho \approx 299\,792\,458 \cdot 40.212 \times 10^{-3} \approx 1.2055 \times 10^7 \text{ m} \approx 12\,055 \text{ km}.$$

This explains why even millisecond-scale delays can map to very large apparent range biases and why the estimator tends to absorb such a common delay into clock-like states rather than into a small geographic displacement.

B SPOOFING-OFFSET BOUNDS GUARD IN EMULATESPOOFING.M

During post-processing, some late epochs contained measurements for satellites whose spoof-difference vector (`Pr_diff_spUser`) was missing or shorter than the set of tracked SVIDs. Without a guard, indexing `Pr_diff_spUser(k)` could go out of bounds and cause the MATLAB script to crash.

To make the tool robust to these missing samples, a minimal bounds check was introduced before using `Pr_diff_spUser(k)`. If k is empty or out of range, the spoof bias is set to 0 for that specific sample; otherwise the original spoof-difference value is used. This keeps the behavior unchanged for samples where spoof-diff exists, while epochs/satellites with missing spoof-diff fall back to a non-spoofed-like contribution instead of causing a hard failure. No plotting functions or plotting flow were modified by this change.

In the implemented code, the guard is realized by defaulting `prBias` to zero and updating it only when k is valid:

```
prBias = 0;
if ~isempty(k) && k <= numel(Pr_diff_spUser)
    prBias = Pr_diff_spUser(k);
end
```